

INF1103: Programming Fundamentals with DevOps

Week 4: File Handling, Arrays and Good Programming Practices

Learning Outcomes

- Store, access, and manipulate data using arrays.
- Read from and write to text files using appropriate file handling techniques
- Apply arrays and file handling together to solve simple real-world problems.
- Develop programs using good programming practices.
- Test and debug programs systematically by validating inputs and handling common errors.



Example Scenario

- Suppose Shopee receives 5 orders.
 - order1 = "Mouse"
 - order2 = "Keyboard"
 - order3 = "USB Cable"
 - order4 = "Monitor"
 - order5 = "Laptop Stand"
- Now imagine Shopee receives 10,000 orders. Should we create?
 - order1
 - order2
 - order3
 - ...
 - order10000

**Impossible to
manage**



Why do we need Arrays?

- Store multiple related values in one variable.
- Manage large amounts of data efficiently.
- Process data using loops.
- Search and analyze information easily.

Arrays (List)

- An array (called a list in Python) allows us to store multiple values in a single variable.
- Instead of creating many variables:
 - `order1 = "Wireless Mouse"`
 - `order2 = "Keyboard"`
 - `order3 = "USB Cable"`
- We can use a list:
 - `orders = ["Wireless Mouse", "Keyboard", "USB Cable"]`
 - The items in a Python list need not be of the same data type.
 - `data = [101, "Alice", 3.8, True]`

List – Concatenation and Repetition

- Concatenation +
 - $[1, 2, 3] + [4, 5, 6]$
- Repetition *
 - $[1, 2, 3]^*4$

List - Index in a sequence

- `my_list[0]`
 - Lists use zero-based indexing.
 - Index 0 refers to the first element.
- `my_list[-1]`
 - Negative indexing counts from the end.
- **Slicing**
 - `my_list[start : end : step]`
- **Membership operator**
 - `in` checks if an item exists in a list
 - `not in` checks if it does not exist.

```
my_list =  
['abc', 123, True, [0, 1, 2], 'fff', -1, -500]  
  
print(my_list)  
print(my_list[0])  
print(my_list[-1])  
print(my_list[:5:2])  
print("abc" in my_list)
```

List operations

- Update:

- `my_list[index] = value` → Update an element

- Add

- `append(value)` → Add to the end
- `insert(index, value)` → Add at a specific position

```
my_list =  
['abc', 123, True, [0, 1, 2], 'fff', -1, -500]  
print(my_list)  
my_list[1] = 'efg'  
print(my_list)  
my_list.append('hij')  
my_list.insert(0, 'klmn')  
print(my_list)
```

List operations

- Delete

- `del my_list[index]` → Remove by index
- `remove(value)` → Remove by value

```
my_list =  
['abc',123,True,[0,1,2],'fff',-1,-500]  
del my_list[-2]  
print(my_list)  
my_list.remove(True)  
print(my_list)  
print(len(my_list))
```

- Length

- `len(my_list)` → Count the number of elements in the list

Loop through a list

- Take each item from the list, one at a time, and do something with it.
- fruit takes on the value of each item in the list one at a time.

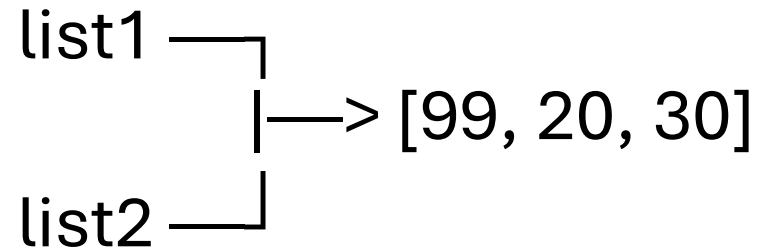
```
fruits = ["Apple", "Banana",  
"Orange"]  
  
for fruit in fruits:  
    print(fruit)
```

List

- Built-in functions

Name	Description
len(list)	Gives the total length of the list.
max(list)	Returns item from the list with max value.
min(list)	Returns item from the list with min value.

Copying Lists



```
list1 = [10, 20, 30]
list2 = list1
list2[0] = 99
print(list1)
print(list2)
```

- does not create a new list.
- Both variables refer to the same list in memory.

Copying Lists

- Copy using `copy()`
 - Now the lists are independent.
- Copy using `list()`
 - Creates a new list containing the same elements.

```
list1 = [10, 20, 30]
list2 = list1.copy()

list2[0] = 99
print(list1)
print(list2)
```

```
list1 = [10, 20, 30]

list2 = list(list1)
print(list1)
print(list2)
```

Copying Lists

- Copy using `copy()`
 - Shallow copy
 - creates a new object but inserts references into it to the objects found in the original.
 - Changes to the copied object can affect the original object if the copied object contains references to mutable objects.
 - Although the outer list was copied, the inner lists are still shared.

```
list1 = [[1, 2], [3, 4]]  
  
list2 = list1.copy()  
  
list2[0][0] = 99  
print(list1)
```

Copying Lists

- Copy using `copy()`
 - Shallow copy
 - Although the outer list was copied, the inner lists are still shared.
- For nested lists:
 - Use `deepCopy()`

```
list1 = [[1, 2], [3, 4]]  
  
list2 = list1.copy()  
  
list2[0][0] = 99  
print(list1)
```

```
import copy  
  
list1 = [[1, 2], [3, 4]]  
  
list2 = copy.deepcopy(list1)  
  
list2[0][0] = 99  
print(list1)
```

Functions stored in a list

- Functions can be stored in a data structure (a list) and executed later
- list 'operations' is storing function objects
- operations[0](5)
 - the first function in operations (add) called with 5

```
def add(x):  
    return x + 10  
  
def multiply(x):  
    return x * 10  
  
def square(x):  
    return x ** 2  
  
operations = [add, multiply, square]  
  
print(operations[0](5))  
print(operations[1](1))  
print(operations[2](5))
```

Learning Outcomes

- Store, access, and manipulate data using arrays.
- **Read from and write to text files using appropriate file handling techniques**
- **Apply arrays and file handling together to solve simple real-world problems.**
- Develop programs using good programming practices.
- Test and debug programs systematically by validating inputs and handling common errors.

Example Scenario



Shopee receives millions of orders every day.
If the application only stores data in variables, all the information
would disappear when the **program closes**.



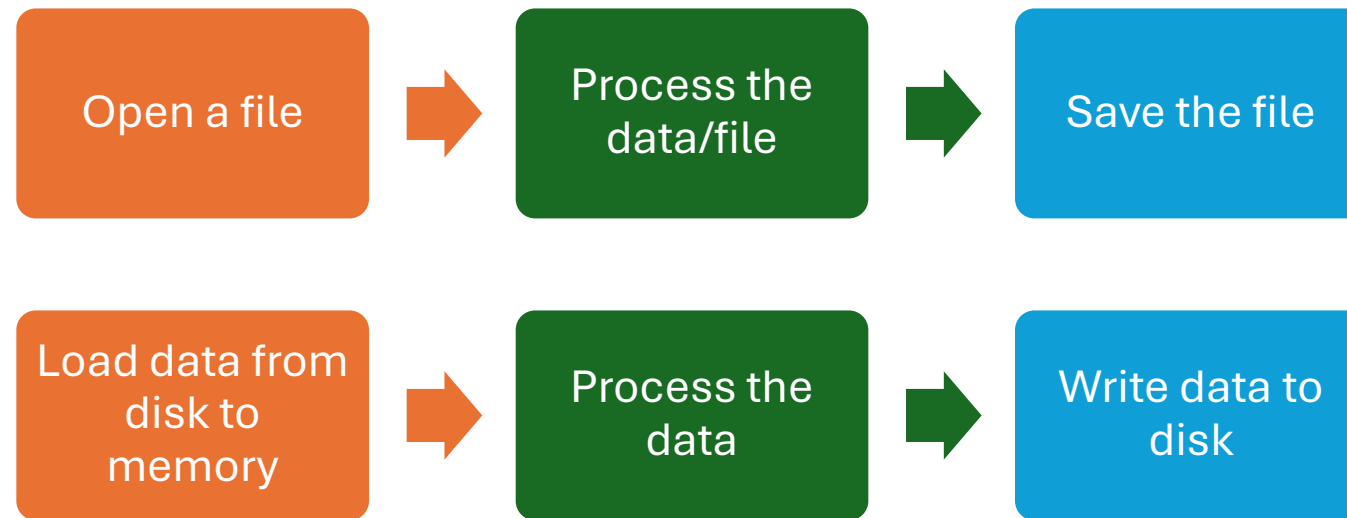
Why do we need File handling?

- Programs lose data when they stop running
- Files provide persistent storage
- Common file types: .txt, .csv, .log, .json
- DevOps uses files for logs, configuration and reports

Why do we need File handling?

- Store **data permanently** so information is not lost when the program stops running.
- **Read existing data** such as customer orders, student records, or server logs.
- Write **new data** such as reports, results, and transaction records.
- **Process large amounts** of data efficiently without requiring users to enter data every time.

File Operations



Open a file

- Python's built-in `open()` function
- Create one **file object** that can be utilized
- Syntax:
 - `file = open(orders.txt, 'r')`
 - `file_object = open(file_name[, access_mode][, encoding])`

Open a file: mode

- `Open('orders.txt', mode='r')`

Mode	Description
r	Opens a file for reading only. This is the default mode.
r+	Opens a file for both reading and writing.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. If the file does not exist, it creates a new file for writing.
b	rb, wb, ab, etc. Opens the file in binary mode.

File pointer

- Indicate the current focused position of the file
 - `open(xxx, mode='r')`
 - The file pointer is placed at the beginning of the file.

Open a file

- with open():
 - Automatically handles file opening and closing
 - Ensures the file is properly closed even if an exception occurs

```
file = open('orders.txt', 'r')  
file.close()
```

```
with open('orders.txt', 'r') as file:  
with open('input.txt', 'r') as infile, open('output.txt', 'w') as outfile:
```

Read a file

- `read()`
 - Reads all contents into a single string
 - Good for small files
 - Not suitable for very large files

```
with open("orders.txt", "r")  
as file:  
    data = file.read()  
  
print(data)
```

Read a file

- `readline(size)`
 - the size parameter specifies the maximum number of characters to read from the line, strictly limiting the read to that number or until the end of the line is reached.
 - Reads the next line each time it is called.
 - Useful when processing records sequentially

```
with open("orders.txt", "r")  
as file:  
    line1 = file.readline()  
    line2 = file.readline()  
  
print(line1)  
print(line2)
```

Read a file

- `readlines(hint)`
 - returns a list containing each line in the file as a list item.
 - the hint parameter suggests the approximate number of characters to read, but it doesn't strictly limit the total number, ensuring complete lines are still returned.
 - Each line becomes an element in a list (array).

```
with open("orders.txt", "r")  
as file:  
    orders = file.readlines()  
  
print(orders)
```

Write data to a file

- `write(str)`
 - Writes a single string
- `writelines(sequence)`
 - Whether the lines include newline characters.
 - Useful when data is already stored in a list.

```
with open("orders.txt", "w") as file:  
    file.write("1001,Wireless Mouse,2\n")  
    file.write("1002,Keyboard,1\n")  
    file.write("1003,USB Cable,3\n")
```

```
orders = [  
    "1001,Wireless Mouse,2\n",  
    "1002,Keyboard,1\n",  
    "1003,USB Cable,3\n"  
]  
  
with open("orders.txt", "w") as file:  
    file.writelines(orders)
```

Append mode

- `Open('orders.txt', mode='a')`

Mode	Description
r	Opens a file for reading only. This is the default mode.
r+	Opens a file for both reading and writing.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. If the file does not exist, it creates a new file for writing.
b	rb, wb, ab, etc. Opens the file in binary mode.
...	

Learning Outcomes

- Store, access, and manipulate data using arrays.
- Read from and write to text files using appropriate file handling techniques
- Apply arrays and file handling together to solve simple real-world problems.
- **Develop programs using good programming practices.**
- **Test and debug programs systematically by validating inputs and handling common errors.**



Why do we need good programming practices?

A program that works is not necessarily a good program.
Good programming practices help us write code that is easier to
read, test, debug, maintain, and improve.

Principle 1: Single Responsibility

- Each function should do one thing well
- Easy to test and debug
- Example: `read_file()`, `calculate_average()`, `display_result()`

Principle 2: DRY

- Don't Repeat Yourself
- Avoid duplicated code
- Reuse functions
- Change logic in one place only



Principle 3: KISS

- Keep it Simple
- Prefer clear solutions over clever ones
- Readability is a feature



Principle 4: IPO Model

- Input → Process → Output
- Provides structure
- Useful for file and array problems



Principle 5: Modular Programming

- Break large problems into small functions and test frequently
- Improves maintainability
- Commit regularly with Git, Supports teamwork
- Supports DevOps culture



Principle 6: Validate

- Validate user inputs
- Check file existence
- Handle unexpected situations
- Detect errors as soon as possible
- Simplifies debugging

Principle 7: Meaningful Naming

- Follow proper naming convention:
 - Eg: student_names not sn
- Improves self-documentation
- Reduces comments needed

Task for the day!!

- Create a file for Shopee Order data
 - Open file
 - read file
 - get max id
 - generate new id
 - append one order
 - write to file





Takeaway!!

- Lists store collections of data
- Files store data permanently
- Lists allow efficient data processing
- Teach habits before frameworks

Thanks!!!

